

Vulnerabilidades implementadas (a propósito) — SecureDevOpsPlayground

Laboratorio Snyk de un solo proyecto: **backend .NET 8** (SecureDevOps.API) +

frontend React (SecureDevOps.Web) + **SQLite** (volumen). App funcional con

vulnerabilidades **controladas y fáciles de resolver**, demostrables con los productos

Snyk: **Code, Open Source, Secrets, IaC y Container**, además de endpoints para

Snyk API & Web (DAST).

Cómo levantar la app

```
docker compose up --build
# Frontend: http://localhost:3000   Backend API: http://localhost:8080
# Usuarios seed: juan@gmail.com / contral234 - admin@gmail.com / Admin123!
```

Cómo correr cada escaneo

```
snyk code test           # Snyk Code (SAST)
snyk test --all-projects # Snyk Open Source
snyk secrets scan .      # Snyk Secrets (fixtures en secrets.example.json)
snyk iac test iac/main.tf # Snyk IaC
snyk container test snyk-lab-backend --file=Dockerfile # Snyk Container (imagen local)
```

1. Snyk Code (SAST) — SecureDevOps.API/

#VulnerabilidadCWEArchivo / punto exactoSeñal Snyk esperadaCómo resolver (fácil) C1**SQL Injection**CWE-89 Services/TaskItemService.cs → SearchAsync (FromSqlRaw con %{query}%)"SQL Injection" (dataflow q → SQL)Usar LINQ: Where(t => t.Title.Contains(query)) C2**Reflected XSS**CWE-79 Controllers/LabController.cs → GET /api/lab/xss (input sin escapar en text/html)"Reflected XSS" / "Improper Neutralization of Input During Web Page Generation"Devolver texto plano o HtmlEncoder.Encode(input) C3**Command Injection**CWE-78 Controllers/LabController.cs → GET /api/lab/ping (host interpolado en /bin/sh -c)"Command Injection"Validar con regex (^[a-z0-9.-]+\$) o lista blanca C4**Path Traversal**CWE-22 Controllers/LabController.cs → GET /api/lab/file (Path.Combine(uploads, name))"Path Traversal" / "Improper Limitation of a Pathname" Path.GetFileName(name) y comprobar prefijo base C5**SSRF**CWE-918 Controllers/LabController.cs → GET /api/lab/fetch (HttpClient.GetStringAsync(url))"Server-Side Request Forgery"Validar URL contra lista blanca de dominios y bloquear IPs privadas C6**Credenciales hardcoded**CWE-798 AppSecrets.cs (constantes AWS/DB/JWT); appsettings.json → Jwt:SecretKey; GET /api/lab/secret las expone"Hardcoded credentials" + "Secret in HTTP response"Leer de variables de entorno / secret manager C7**Broken Object Level Authorization (IDOR)**CWE-639 Services/TaskItemService.cs (todas las consultas) + Controllers/TaskItemController.cs (usa

`dto.CreatedByUserId`) "Improper Authorization" / "Broken Access Control" Filtrar por `ClaimTypes.NameIdentifier` del token

Fácil de resolver = cada fix es 1–3 líneas y no rompe la funcionalidad.

2. Snky Open Source — `SecureDevOps.Web/package.json`

Versiones ANCIANAS de paquetes con CVEs conocidos que Snky Open Source detecta al escanear `--all-projects` (y que npm reporta en el build → **5 vulns: 4 high, 1 critical**):

PaqueteVersión (actual en el repo)Hallazgo esperadoFix `lodash` `^4.17.15` Prototype Pollution (CVE-2019-10744, CVE-2021-23337) `^4.17.21` (o última) `minimist` (dev) `^1.2.0` Prototype Pollution (CVE-2020-7598, CVE-2021-44906) `^1.2.8` `yargs-parser` (dev) `^5.0.1` Prototype Pollution (CVE-2020-7608)Última 5.x/21.x

3. Snky Secrets — `secrets.example.json` + código

FixtureValor (ficticio)Regla Snky Secrets AWS Access Key `AKIAIOSFODNN7EXAMPLE` AWS access key AWS Secret Key `wJa1rXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY` AWS secret key DB password `P@ssw0rd_Demo_123!` Generic secret JWT secret `SuperSecretJwtKeyHardcoded_Demo_123!` Generic secret Private key `-----BEGIN PRIVATE KEY-----...` Private key

También detecta las constantes de `AppSecrets.cs` y `appsettings.json`. Fix: borrar fichero + pasar credenciales por variables de entorno; rotar si se filtraron.

4. Snky IaC — `iac/main.tf`

#HallazgoRegla Snky IaC esperadaFix I1S3 con ACL `public-read` S3 Bucket ACL All Users Read `acl = "private"` + bloqueo de acceso público I2SG abierto `0.0.0.0/0` en 22 y 3306 SSH Port Open to Internet, MySQL open to the internet CIDR restringido / referenciar SG I3RDS `storage_encrypted=false` RDS Storage Unencrypted `storage_encrypted = true` I4RDS `backup_retention_period=0` RDS Backup Disabled `>= 7` I5RDS `publicly_accessible=true` RDS Publicly Accessible `false` + subredes privadas I6RDS con password hardcodeada `PlainText` Password variables `sensitive` I7IAM `Action:["*"] / Resource:["*"]` IAM policy grants `all access` policies de menor privilegio I8Credenciales en el provider `PlainText` (AWS keys) `~/.aws/credentials` / OIDC

5. Snky Container — `Dockerfile` (backend)

HallazgoRegla Snky Container esperadaFix Runtime = imagen `dotnet/sdk:8.0` (trae build tools y más CVEs)CVEs de alta gravedad en la imagen base (Debian bookworm + SDK)Multi-stage con `aspnet:8.0` como runtime Ejecución como **root**"Image runs as root user" `USER app` (imagen aspnet trae `app`) Sin `HEALTHCHECK` Recomendación de base `HEALTHCHECK CMD curl -fs http://localhost:8080/health`

6. Snky API & Web (DAST) — App en ejecución

Los escáneres dinámicos (Snky API & Web) descubren estas rutas en el frontend y las prueban con payloads:

EndpointMétodoDescripción / payload sugerido

/api/auth/register	POST	Body {"username","email","password","firstName","lastName"}
/api/auth/login	POST	Body {"email","password"} → token JWT
/api/auth/logout	POST	Cierre de sesión
/api/user/profile	GET	Perfil autenticado
/api/taskitem	GET/POST	Listar / crear tareas
/api/taskitem/{id}	GET/PUT/DELETE	Tarea por id (IDOR: juega con ids ajenos)
/api/taskitem/search?q=	GET	SQLi: q=' UNION SELECT ...--
/api/lab/xss?input=	GET	<script>alert(1)</script>
/api/lab/search?q=	GET	q=' OR '1'='1
/api/lab/file?name=	GET	name=../../etc/passwd
/api/lab/ping?host=	GET	host=localhost;whoami
/api/lab/fetch?url=	GET	url=http://169.254.169.254/latest/meta-data/
/api/lab/redirect?url=	GET	url=https://evil.com
/api/lab/secret	GET	Expone secretos hardcodeados

Resumen (tabla única)

IDProducto SnykArchivoGravedad esperada C1CodeTaskItemService.csHigh C2CodeLabController.cs (xss)Medium/High C3CodeLabController.cs (ping)High C4CodeLabController.cs (file)Medium/High C5CodeLabController.cs (fetch)Medium C6Code/SecretsAppSecrets.cs, appsettings.jsonHigh/Medium C7CodeTaskItemService/ControllerHigh O1–O3Open Sourcepackage.json (lodash, minimist, yargs-parser)High/Critical S1–S5Secretssecrets.example.jsonHigh/Medium I1–I8IaCiac/main.tfHigh/Critical D1–D4ContainerDockerfileHigh + recomendaciones —API & WebEndpoints

/api/*
/api/lab/*

Array (DAST)